ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Lauren Shohan

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```
#Load/install required package here
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(ggplot2)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(Kendall)
library(tseries)
library(outliers)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.5.1
## v forcats 1.0.0     v tibble  3.2.1
## v purrr   1.0.2     v tidyr   1.3.1
## v readr   2.1.5
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```r
#install.packages("smooth")
library(smooth)
```

```
## Loading required package: greybox
## Package "greybox", v2.0.8 loaded.
##
##
## Attaching package: 'greybox'
##
## The following object is masked from 'package:tidyr':
##
##     spread
##
## The following object is masked from 'package:lubridate':
##
##     hm
##
## This is package "smooth", v4.4.0
```

Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```
#Importing time series data from text file
NetGen_data <- read.csv(file="/home/guest/TSA_Sp26/Data/Net_generation_United_States_all_sectors_monthly
#Inspect data
head(NetGen_data)
```

```
##      Month all.fuels..utility.scale..thousand.megawatthours
## 1 Dec 2020                                          344970.4
## 2 Nov 2020                                          302701.8
## 3 Oct 2020                                          313910.0
## 4 Sep 2020                                          334270.1
## 5 Aug 2020                                          399504.2
## 6 Jul 2020                                          414242.5
##   coal.thousand.megawatthours natural.gas.thousand.megawatthours
## 1                    78700.33                            125703.7
## 2                    61332.26                            109037.2
## 3                    59894.57                            131658.2
## 4                    68448.00                            141452.7
## 5                    91252.48                            173926.6
## 6                    89831.36                            185444.8
##   nuclear.thousand.megawatthours
## 1                       69870.98
## 2                       61759.98
## 3                       59362.46
## 4                       65727.32
## 5                       68982.19
## 6                       69385.44
##   conventional.hydroelectric.thousand.megawatthours
## 1                                          23086.37
## 2                                          21831.88
## 3                                          18320.72
## 4                                          19161.97
## 5                                          24081.57
## 6                                          27675.94
```

```
nvar <- ncol(NetGen_data) - 1
nobs <- nrow(NetGen_data)

#Preparing the data - create date object and rename columns
NG_NetGen_processed <-
  NetGen_data %>%
  mutate( Month = my(Month) ) %>%
  rename(Natural_Gas = natural.gas.thousand.megawatthours ) %>%
  select(1,4) %>%
  arrange( Month )

head(NG_NetGen_processed)
```

```
##        Month Natural_Gas
```

```
## 1 2001-01-01    42388.66
## 2 2001-02-01    37966.93
## 3 2001-03-01    44364.41
## 4 2001-04-01    45842.75
## 5 2001-05-01    50934.21
## 6 2001-06-01    57603.15
```

```
summary(NG_NetGen_processed)
```

```
##      Month             Natural_Gas
##  Min.   :2001-01-01   Min.   : 37967
##  1st Qu.:2005-12-24   1st Qu.: 62245
##  Median :2010-12-16   Median : 84415
##  Mean   :2010-12-16   Mean   : 88028
##  3rd Qu.:2015-12-08   3rd Qu.:108385
##  Max.   :2020-12-01   Max.   :185445
```

```
#No NAs so we don't need to worry about missing values
```

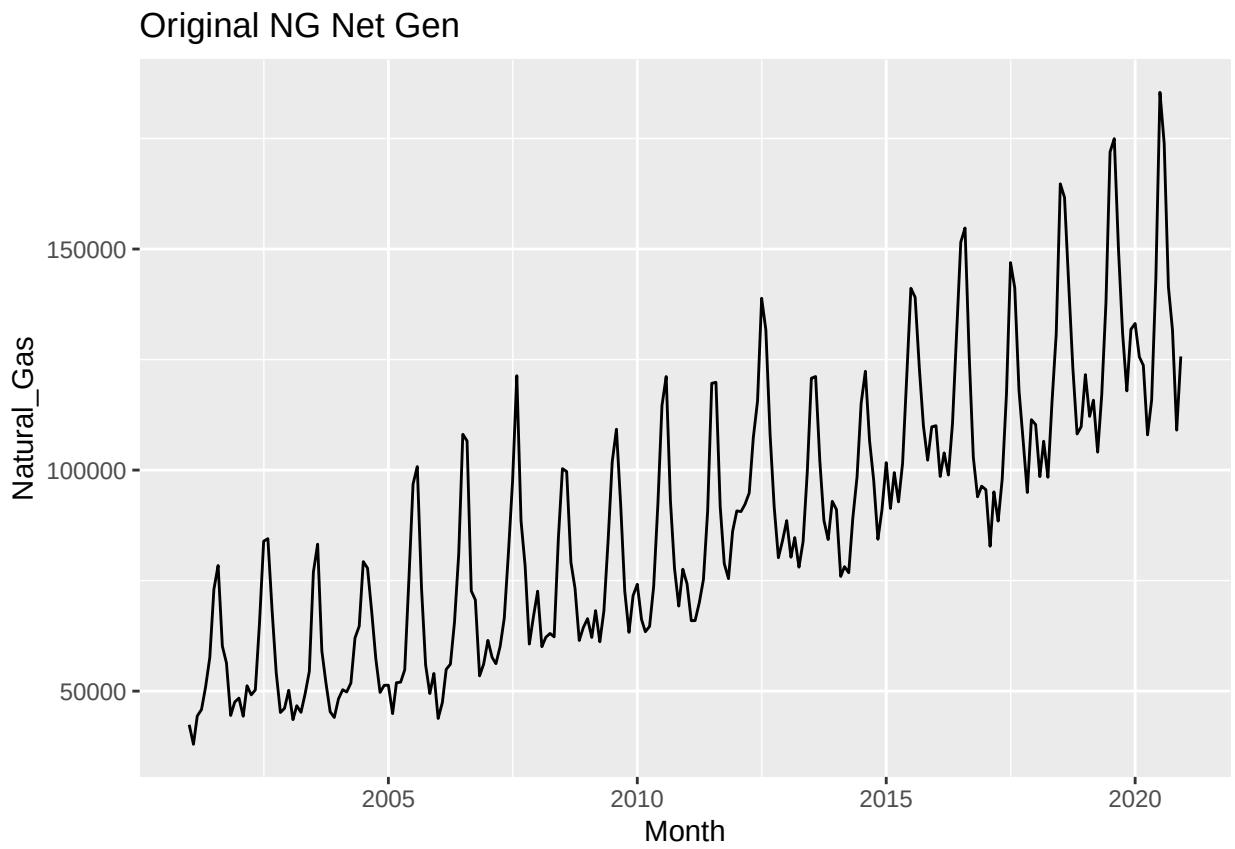
```
ts_naturalgas <- ts(NG_NetGen_processed[,2],
                          start=c(year(NG_NetGen_processed$Month[1]),month(NG_NetGen_processed$Month[1
                          frequency=12)
#note that we are only transforming columns with netgen mw, not the date columns
head(ts_naturalgas,15)
```

```
##           Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 2001 42388.66 37966.93 44364.41 45842.75 50934.21 57603.15 73030.14 78409.80
## 2002 48412.83 44308.43 51214.46
##           Sep      Oct      Nov      Dec
## 2001 60181.14 56376.44 44490.62 47540.86
## 2002
```

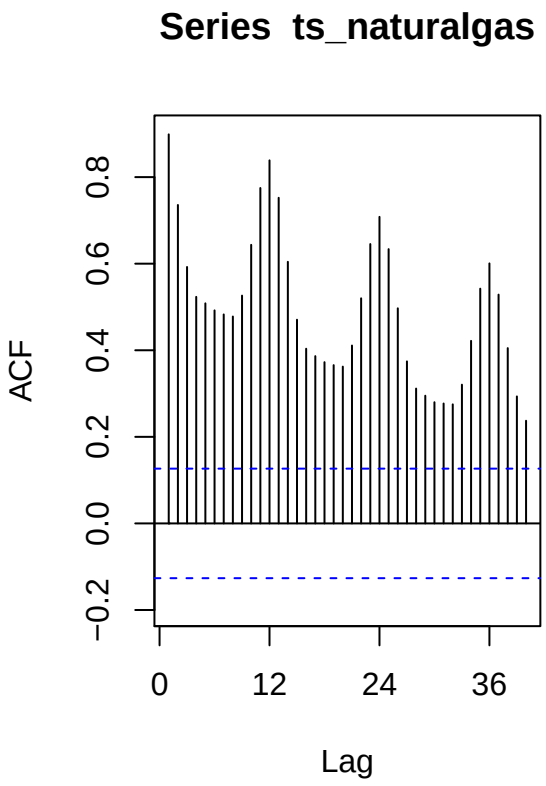
```
tail(ts_naturalgas,15)
```

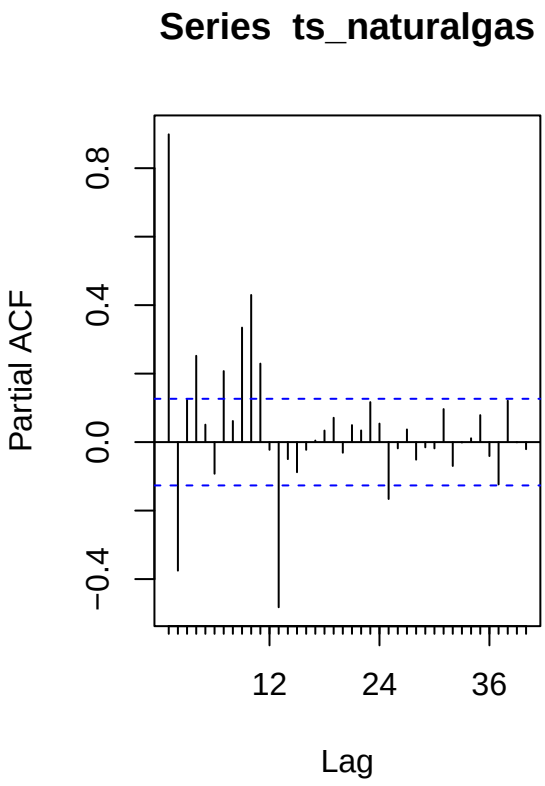
```
##           Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 2019
## 2020 133157.6 125593.9 123697.0 107960.0 115870.9 143245.4 185444.8 173926.6
##           Sep      Oct      Nov      Dec
## 2019          130947.6 117910.5 131838.9
## 2020 141452.7 131658.2 109037.2 125703.7
```

```
TS_Plot <-
  ggplot(NG_NetGen_processed, aes(x=Month, y=Natural_Gas)) +
  ggtitle('Original NG Net Gen') +
      geom_line()
plot(TS_Plot)
```




```
#ACF and PACF plots
par(mfrow=c(1,2))
ACF_Plot <- Acf(ts_naturalgas, lag = 40, plot = TRUE)
PACF_Plot <- Pacf(ts_naturalgas, lag = 40)
```



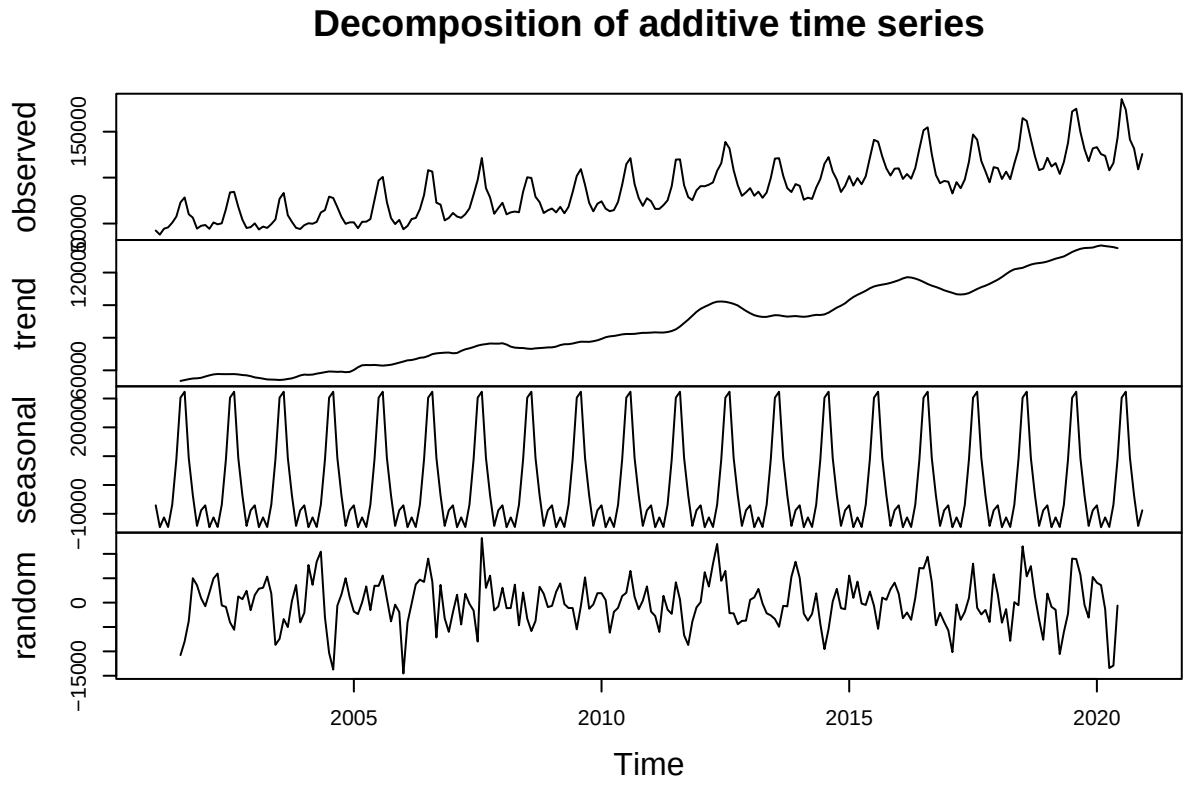



```
par(mfrow=c(1,1))
```

Q2

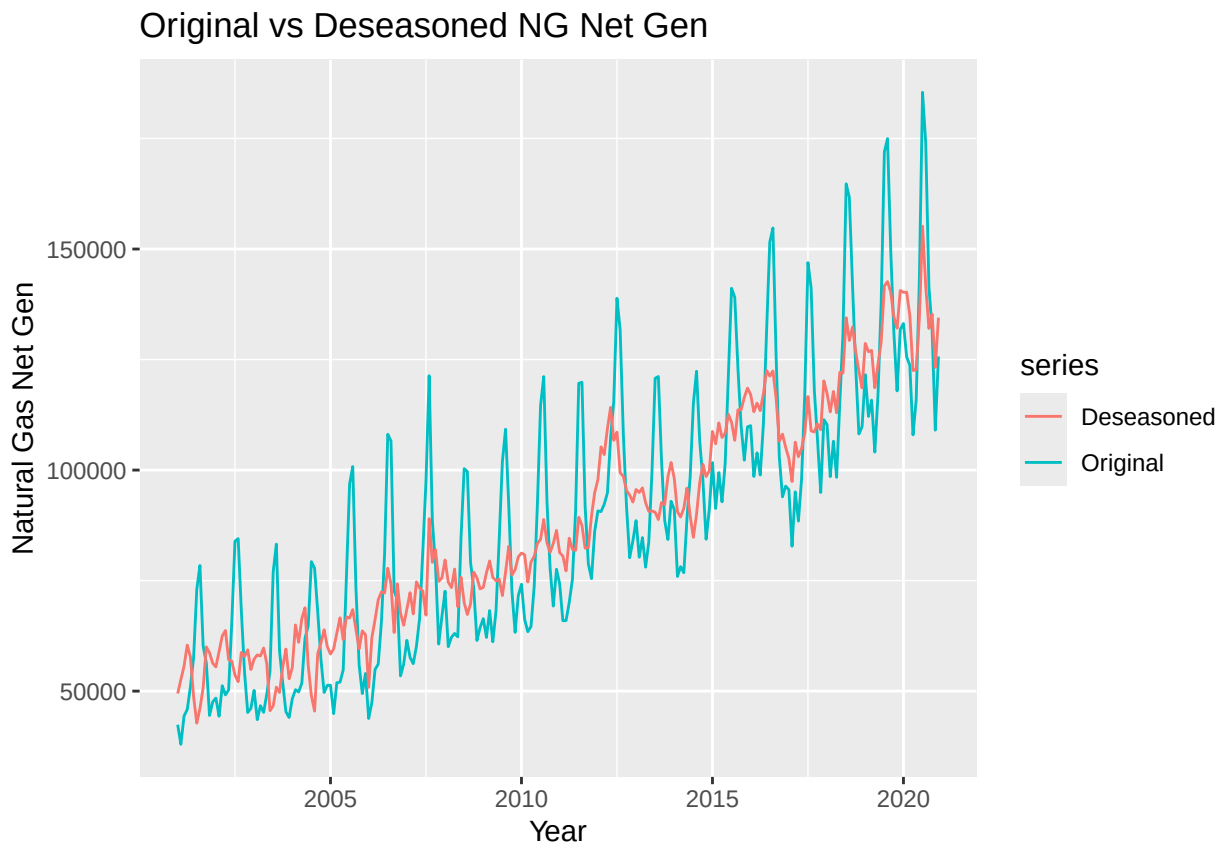
Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

```
#luana M7 notes i am just copying and pasting for reference later -

#The plots from the previous section show the data has a seasonal component. Since we are working with

#Using R decompose function
decompose_NG <- decompose(ts_naturalgas,"additive")
plot(decompose_NG)
```



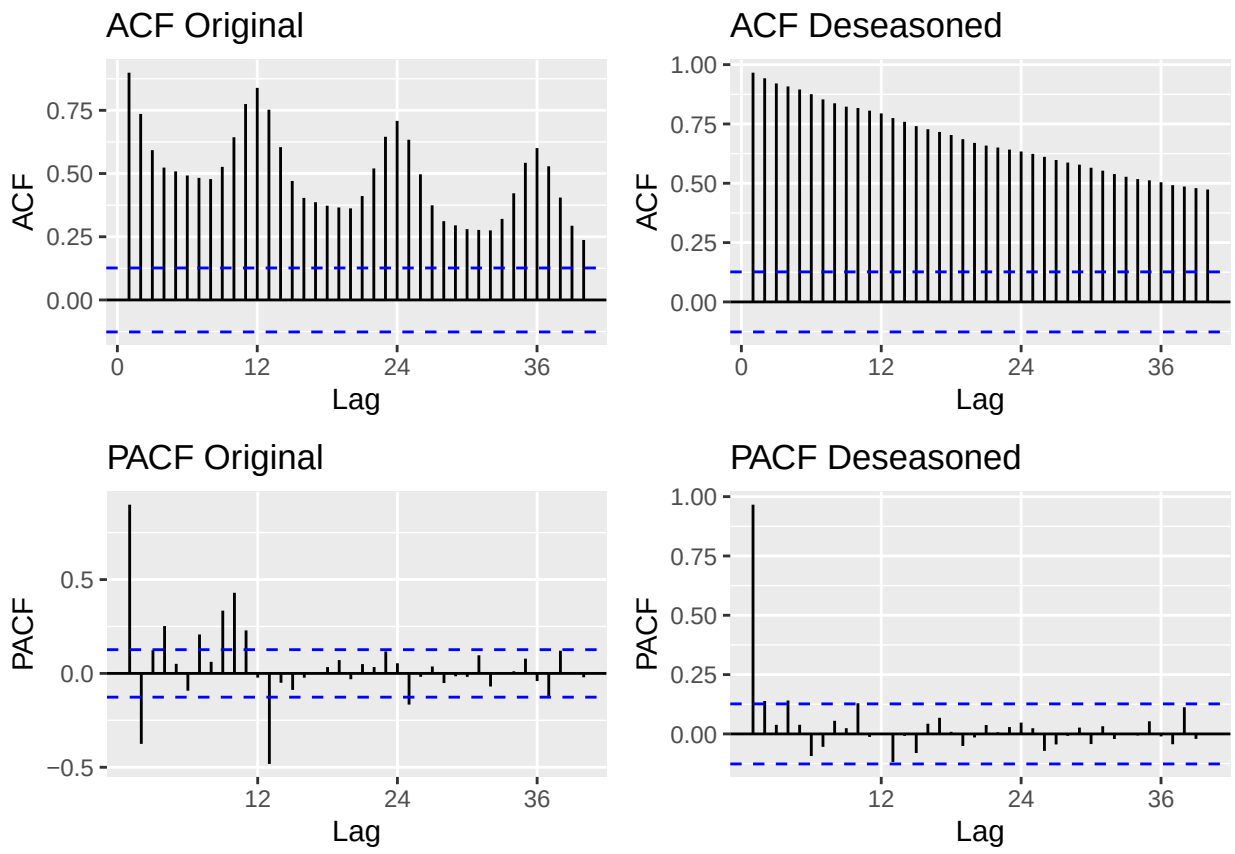

```
#The ACF plot show a slow decay which is a sign of non-stationarity.

#Creating non-seasonal residential price time series because some models can't handle seasonality
deseasonal_NG <- seasadj(decompose_NG)
```

```
#deseasoned over time
autoplot(ts_naturalgas, series = 'Original') +
  autolayer(deseasonal_NG, series = 'Deseasoned') +
  xlab('Year') + ylab('Natural Gas Net Gen') +
  ggtitle('Original vs Deseasoned NG Net Gen')
```




```
#comparing acf and pacf
plot_grid(
  autoplot(Acf(ts_naturalgas, lag = 40, plot = FALSE),
           main = 'ACF Original'),
  autoplot(Acf(deseasonal_NG, lag = 40, plot = FALSE),
           main = 'ACF Deseasoned'),
  autoplot(Pacf(ts_naturalgas, lag = 40, plot = FALSE),
           main = 'PACF Original'),
  autoplot(Pacf(deseasonal_NG, lag = 40, plot = FALSE),
           main = 'PACF Deseasoned'),
  nrow = 2)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: ‘main‘
## Ignoring unknown parameters: ‘main‘
## Ignoring unknown parameters: ‘main‘
## Ignoring unknown parameters: ‘main‘
```



>Answer: We can see the ACF deseasoned doesnt have seasonality spikes any longer but it still slowly decaying at same rate, the slow decay showcses nonstationary. PACF deseasoned also got rid of the large spikes after the first lag. The fluctations of the series over time also diminish and grow have less distance between them.

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```
#adf
print(adf.test(deseasonal_NG, alternative = 'stationary'))
```

```
## Warning in adf.test(deseasonal_NG, alternative = "stationary"): p-value smaller
## than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  deseasonal_NG
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
#mann kendall
print(MannKendall(deseasonal_NG))
```

```
## tau = 0.843, 2-sided pvalue =< 2.22e-16
```

> Answer: The ADF resulted in an pvalue of 0.01 so I reject the null hypothesis that the series has unit root and can conclude the series is stationary. MannKendall resulted in tau = 0.843, 2-sided pvalue =< 2.22e-16. Thus, we reject the null hypothesis and conclude there is significant deterministic upward trend as we can see in the deseasoned graph from Q2.

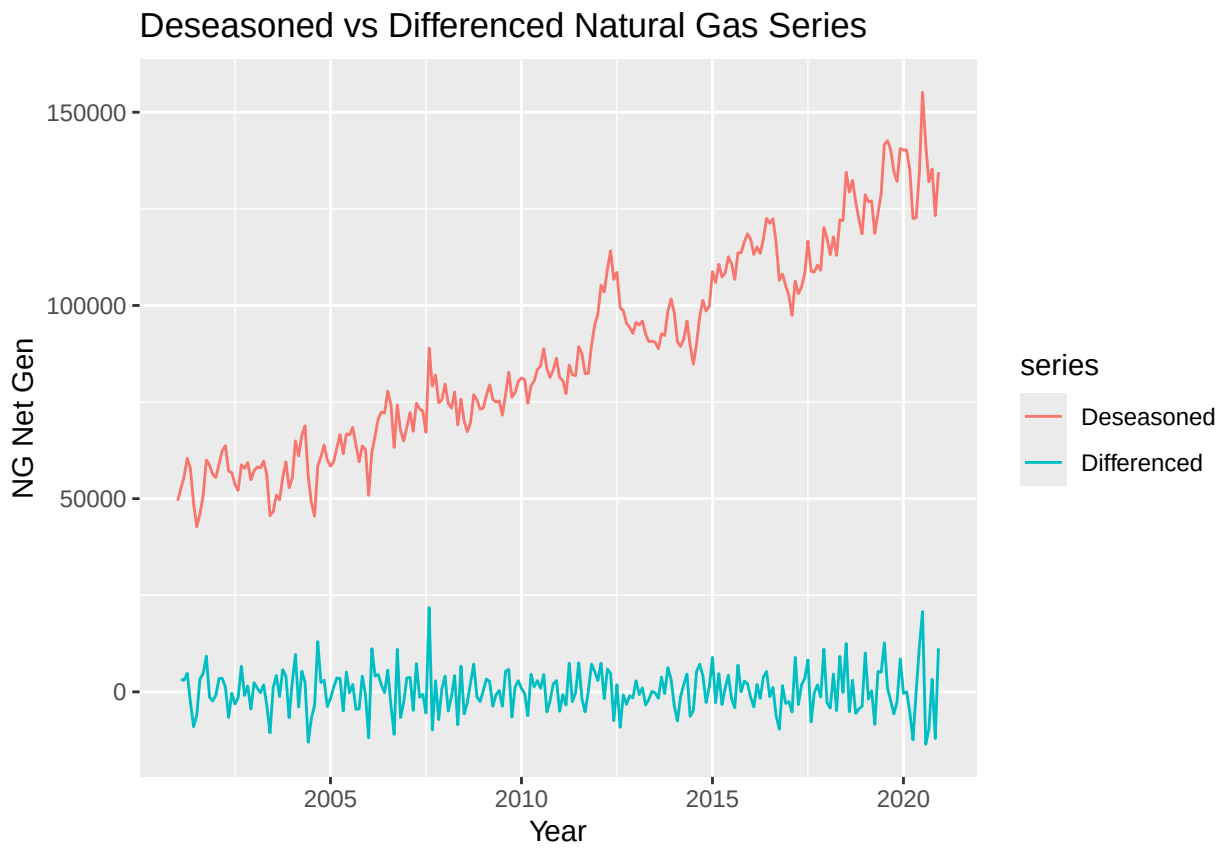
Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

```r
#first i will determine d - ndiffs() to see how many times to difference
n_diff <- ndiffs(deseasonal_NG)
n_diff
```

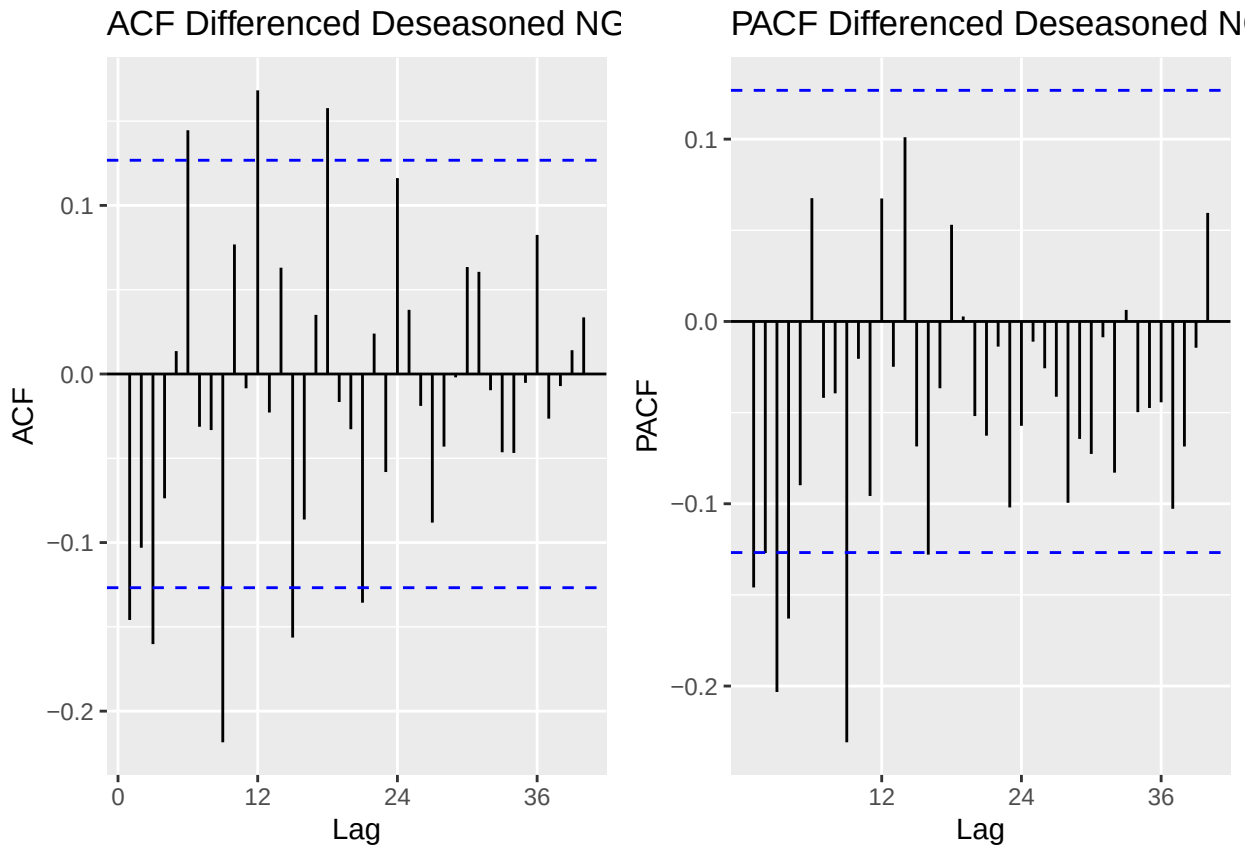
```
## [1] 1
```

```r
#difference deseasoned series
deseasonal_NG_diff <- diff(deseasonal_NG, differences = 1)

#plot
autoplot(deseasonal_NG, series = 'Deseasoned') +
  autolayer(deseasonal_NG_diff, series = 'Differenced') +
  xlab('Year') + ylab('NG Net Gen') +
  ggtitle('Deseasoned vs Differenced Natural Gas Series')
```




```
# Check ACF and PACF of differenced series to identify p and q
plot_grid(
  autoplot(Acf(deseasonal_NG_diff, lag.max = 40, plot = FALSE),
           main = 'ACF Differenced Deseasoned NG'),
  autoplot(Pacf(deseasonal_NG_diff, lag.max = 40, plot = FALSE),
           main = 'PACF Differenced Deseasoned NG'),
  nrow = 1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: ‘main‘
## Ignoring unknown parameters: ‘main‘
```



>Answer: After differencing the series by 1, we can see on the plot by the blue line around 0 that this was the correct route to pursue!The blue differenced series around zero has no upward trend compared to the deaseasoned trend climbing upward. After differencing, the ACF and PACF have basically no lags (except a small one that just goes past the bounds) and looks like white noise which we can infer as the AR/MA doesnt need much beyond the differencing. This means we can say it could be ARIMA(0,1,0):

- p=0 no sig AR visible in PACF after differencing
- d=1 one difference confirmed
- q=0 no sig MA structure visible in ACF after differencing

additional functions - The few significant spikes visible in the plots could suggest trying ARIMA(1,1,0) or ARIMA(0,1,1) as alternative candidates to compare

Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```
#ARIMA(0,1,0) with drift
Model_010 <- Arima(deseasonal_NG, order = c(0,1,0), include.drift = TRUE)
print(Model_010)
```

```
## Series: deseasonal_NG
## ARIMA(0,1,0) with drift
```

```
##
## Coefficients:
##          drift
##       355.7871
## s.e.  358.1986
##
## sigma^2 = 30794133:  log likelihood = -2399.14
## AIC=4802.29   AICc=4802.34   BIC=4809.24
```

```
print(Model_010$coef)
```

```
##    drift
## 355.7871
```

```
#ARIMA(1,1,0) and ARIMA(0,1,1) alternatives
Model_110 <- Arima(deseasonal_NG, order = c(1,1,0), include.drift = TRUE)
print(Model_110)
```

```
## Series: deseasonal_NG
## ARIMA(1,1,0) with drift
##
## Coefficients:
##           ar1     drift
##       -0.1479  348.3927
## s.e.   0.0644  308.8385
##
## sigma^2 = 30254066:  log likelihood = -2396.54
## AIC=4799.07   AICc=4799.18   BIC=4809.5
```

```
print(Model_110$coef)
```

```
##         ar1       drift
##  -0.1478609 348.3926570
```

```
Model_011 <- Arima(deseasonal_NG, order = c(0,1,1), include.drift = TRUE)
print(Model_011)
```

```
## Series: deseasonal_NG
## ARIMA(0,1,1) with drift
##
## Coefficients:
##           ma1     drift
##       -0.2296  344.5131
## s.e.   0.0866  271.9198
##
## sigma^2 = 29946343:  log likelihood = -2395.33
## AIC=4796.66   AICc=4796.77   BIC=4807.09
```

```
print(Model_011$coef)
```

```
##         ma1       drift
##  -0.2296014 344.5131362
```

```
# Compare AIC across all three models
cat('AIC Comparison:\\n')
```

```
## AIC Comparison:\n
```

```
cat('ARIMA(0,1,0):', Model_010$aic, '\\n')
```

```
## ARIMA(0,1,0): 4802.288 \n
```

```
cat('ARIMA(1,1,0):', Model_110$aic, '\\n')
```

```
## ARIMA(1,1,0): 4799.075 \n
```

```
cat('ARIMA(0,1,1):', Model_011$aic, '\\n')
```

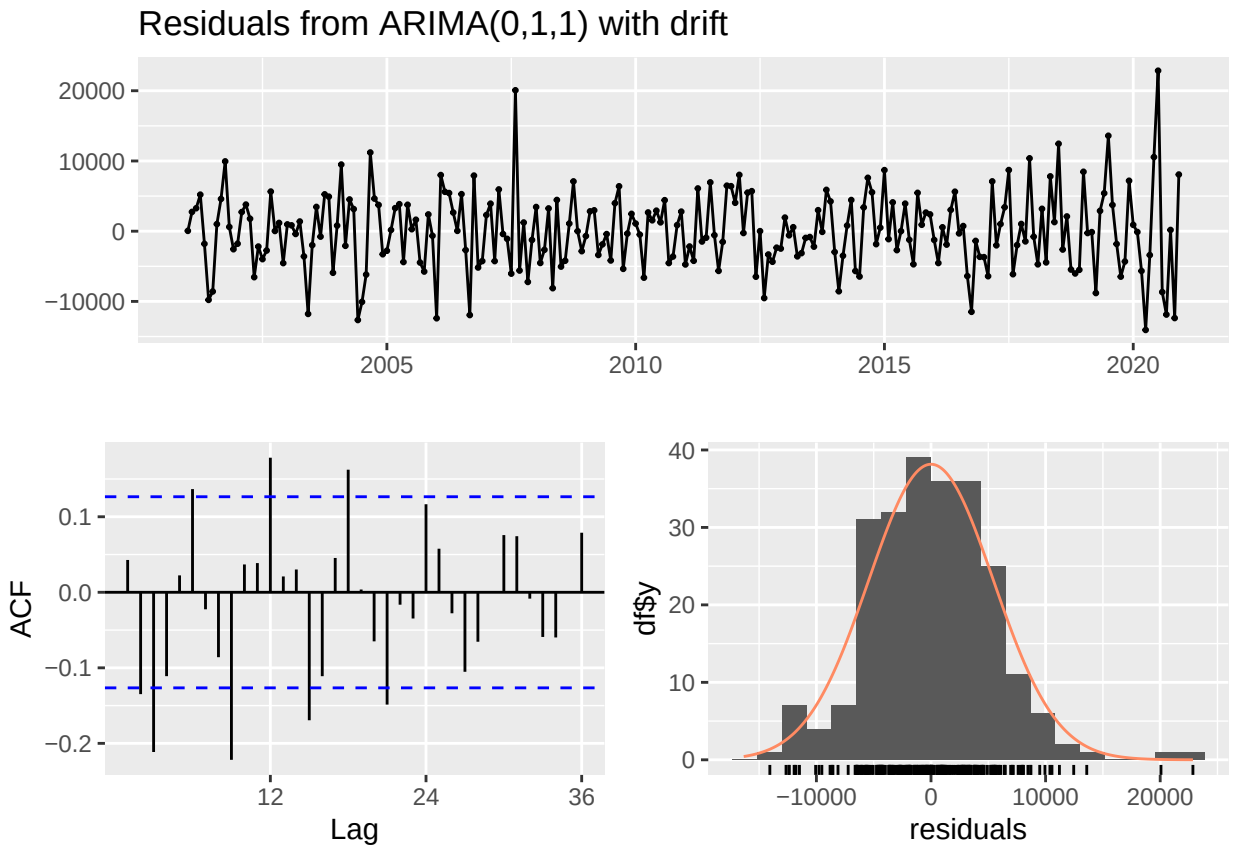
```
## ARIMA(0,1,1): 4796.664 \n
```

> Answer: ARIMA(0,1,1) with drift is the best model because it has the lowest AIC of 4796.66. The drift coefficient of ~344-356 is consistent across all three models, confirming the strong upward trend in natural gas generation.

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

```
checkresiduals(Model_011)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1) with drift
## Q* = 76.2, df = 23, p-value = 1.295e-07
##
## Model df: 1.   Total lags used: 24
```

Answer: yes the residuals of 0,1,1, look like white noise. They fluctate around zero with no visible trend and the variance seems constant over time with a few notable spikes (one likely due to covid).The ACF mostly falls in bounds which is what white noise looks like while the histogram is normally distributed and centered around 0. Thus, all centered around zero, normally distributed, and uncorrelated is white noise and shows the model is a decent fit.

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

Q7 part Q3

Run the ADF test and Mann Kendall test - Report and explain the results.

```r
#adf
print(adf.test(ts_naturalgas, alternative = 'stationary'))
```

```
## Warning in adf.test(ts_naturalgas, alternative = "stationary"): p-value smaller
## than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  ts_naturalgas
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
#mann kendall
print(MannKendall(ts_naturalgas))
```

```
## tau = 0.651, 2-sided pvalue =< 2.22e-16
```

> Answer: The ADF resulted in an pvalue of 0.01 so I reject the null hypothesis that the series has unit root and can conclude the series is stationary. MannKendall resulted in tau = 0.651, 2-sided pvalue =< 2.22e-16. Thus, we reject the null hypothesis and conclude there is significant deterministic upward trend as we can see in the deseasoned graph from Q2.

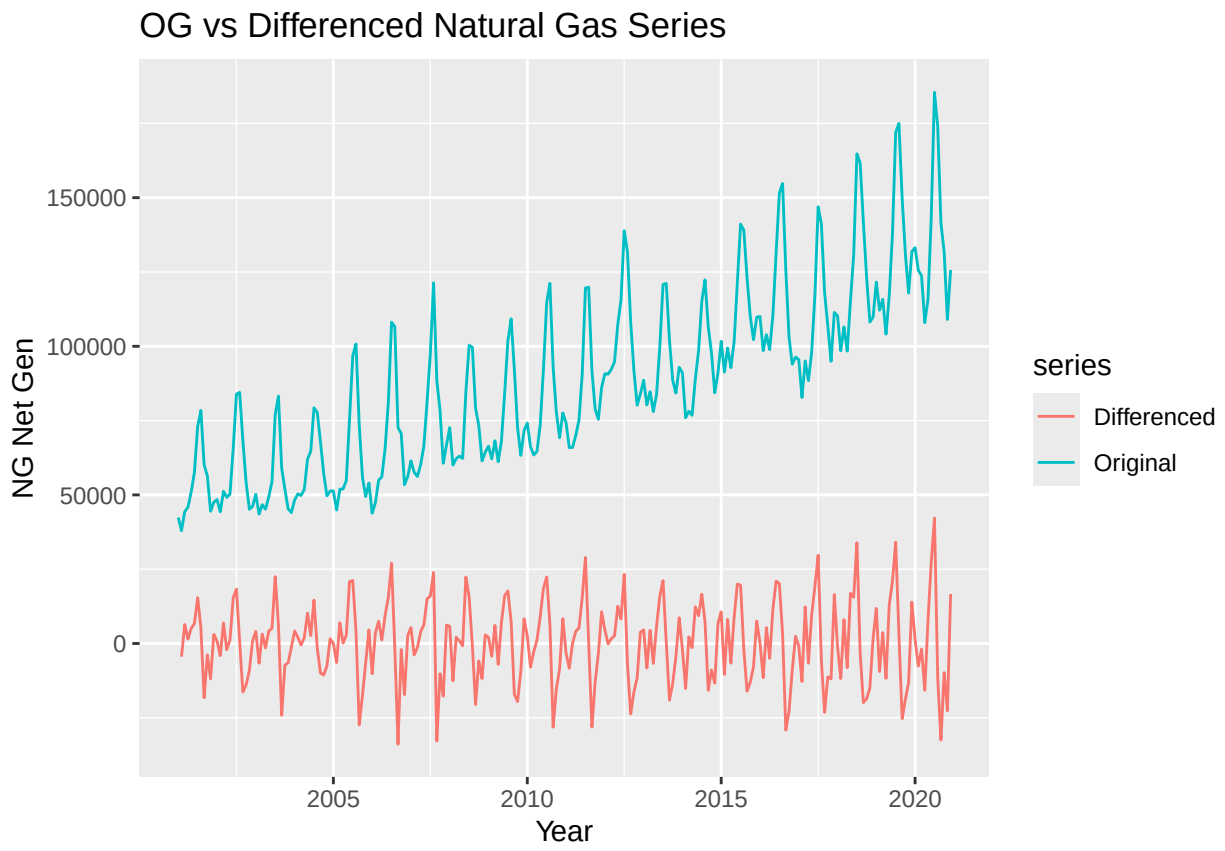
Q7 - part Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

```r
#first i will determine d - nsdiffs() to see how many times to difference
n_diff2 <- ndiffs(ts_naturalgas)
n_diff2
```

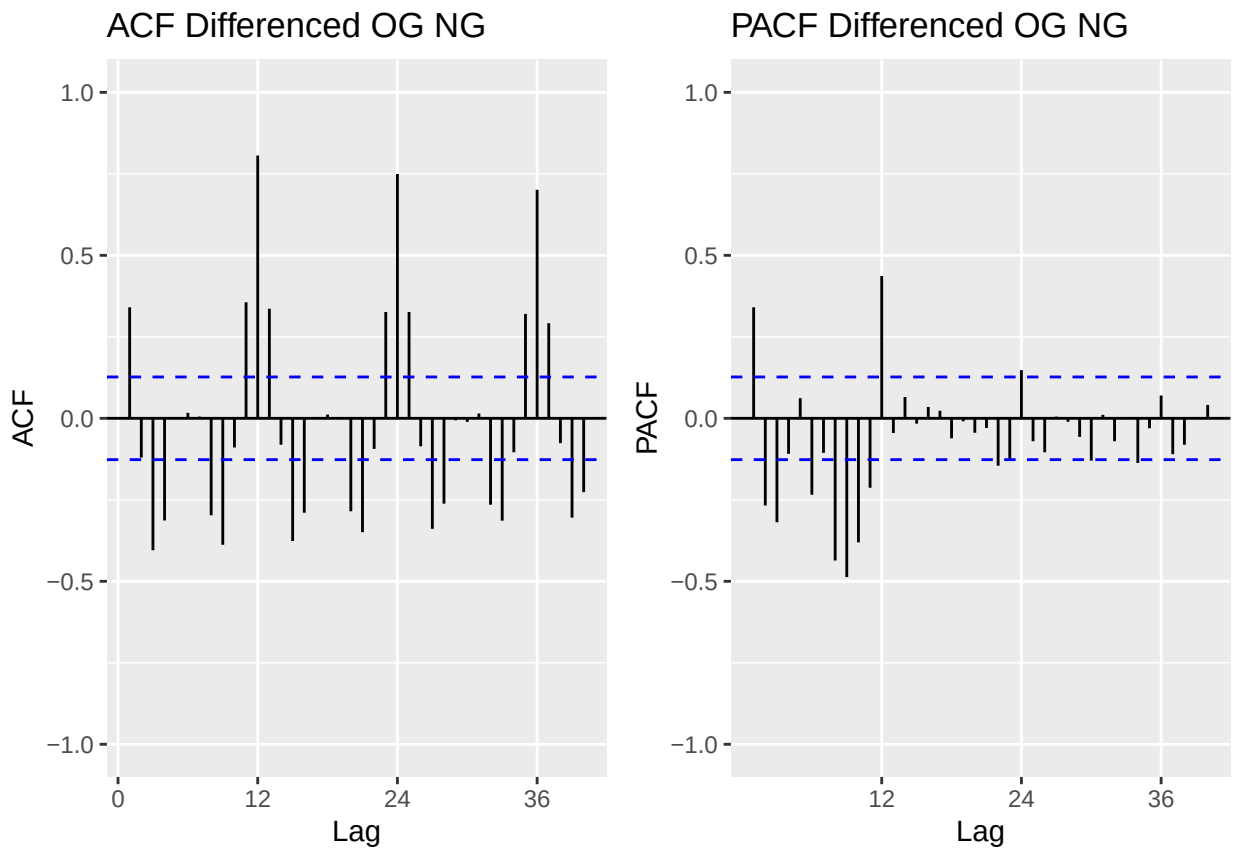
```
## [1] 1
```

```r
#difference og series
OG_NG_diff <- diff(ts_naturalgas, differences = 1)

#plot
autoplot(ts_naturalgas, series = 'Original') +
  autolayer(OG_NG_diff, series = 'Differenced') +
  xlab('Year') + ylab('NG Net Gen') +
  ggtitle('OG vs Differenced Natural Gas Series')
```




```
# Check ACF and PACF of differenced series to identify p and q
plot_grid(
  autoplot(Acf(OG_NG_diff, lag.max = 40, plot = FALSE),
           main = 'ACF Differenced OG NG', ylim = c(-1,1)),
  autoplot(Pacf(OG_NG_diff, lag.max = 40, plot = FALSE),
           main = 'PACF Differenced OG NG',ylim = c(-1,1)),
  nrow = 1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: ‘main‘ and ‘yli
## Ignoring unknown parameters: ‘main‘ and ‘ylim‘
```



>Answer: After trend differencing original seasonal peaks and dips are still visible in the pink differenced series — the regular spikes repeating every 12 months confirm that one round of trend differencing alone is not enough This is why D=1 seasonal differencing is also needed. The large spikes at seasonal lags 12, 24, 36 in the ACF confirm remaining seasonality even after trend differencing. The ACF seasonal spikes are tailing off gradually across lags 12, 24, 36 which is consistent with a seasonal AR process. The PACF shows a sharp large spike at lag 12 and smaller spikes at lags 24 and beyond that decay which is toward a seasonal AR(1) component. The non-seasonal lags show a negative spike at lag 1 in the ACF which leans toward a non-seasonal MA(1) term.

Q7 - part Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```
Model_011_011 <- Arima(ts_naturalgas, order = c(0,1,1), seasonal = c(0,1,1), include.drift = TRUE)
```

```
## Warning in Arima(ts_naturalgas, order = c(0, 1, 1), seasonal = c(0, 1, 1), : No
## drift term fitted as the order of difference is 2 or more.
```

```
print(Model_011_011)
```

```
## Series: ts_naturalgas
## ARIMA(0,1,1)(0,1,1)[12]
##
```

```
## Coefficients:
##           ma1     sma1
##       -0.2694  -0.6845
## s.e.   0.0784   0.0563
##
## sigma^2 = 30164374:  log likelihood = -2279.64
## AIC=4565.28   AICc=4565.39   BIC=4575.56
```

```
print(Model_011_011$coef)
```

```
##        ma1       sma1
## -0.2694132 -0.6845094
```

```
#ARIMA(1,1,0) and ARIMA(0,1,1) alternatives
Model_010_011 <- Arima(ts_naturalgas, order = c(0,1,0), seasonal = c(0,1,1), include.drift = TRUE)
```

```
## Warning in Arima(ts_naturalgas, order = c(0, 1, 0), seasonal = c(0, 1, 1), : No
## drift term fitted as the order of difference is 2 or more.
```

```
print(Model_010_011)
```

```
## Series: ts_naturalgas
## ARIMA(0,1,0)(0,1,1)[12]
##
## Coefficients:
##          sma1
##       -0.6982
## s.e.   0.0557
##
## sigma^2 = 31481438:  log likelihood = -2285.18
## AIC=4574.35   AICc=4574.4   BIC=4581.2
```

```
print(Model_010_011$coef)
```

```
##      sma1
## -0.698203
```

```
Model_110_011 <- Arima(ts_naturalgas, order = c(1,1,0), seasonal = c(0,1,1), include.drift = TRUE)
```

```
## Warning in Arima(ts_naturalgas, order = c(1, 1, 0), seasonal = c(0, 1, 1), : No
## drift term fitted as the order of difference is 2 or more.
```

```
print(Model_110_011)
```

```
## Series: ts_naturalgas
## ARIMA(1,1,0)(0,1,1)[12]
##
## Coefficients:
##           ar1     sma1
```

```
##       -0.1808  -0.6898
## s.e.   0.0655   0.0557
##
## sigma^2 = 30626308:  log likelihood = -2281.43
## AIC=4568.86   AICc=4568.96   BIC=4579.13
```

```
print(Model_110_011$coef)
```

```
##        ar1       sma1
## -0.1808294 -0.6897961
```

```
# Compare AIC across all three models
cat('AIC Comparison:\\n')
```

```
## AIC Comparison:\n
```

```
cat('SARIMA _011_011:', Model_011_011$aic, '\\n')
```

```
## SARIMA _011_011: 4565.282 \n
```

```
cat('SARIMA _010_011:', Model_010_011$aic, '\\n')
```

```
## SARIMA _010_011: 4574.35 \n
```

```
cat('SARIMA _110_011:', Model_110_011$aic, '\\n')
```

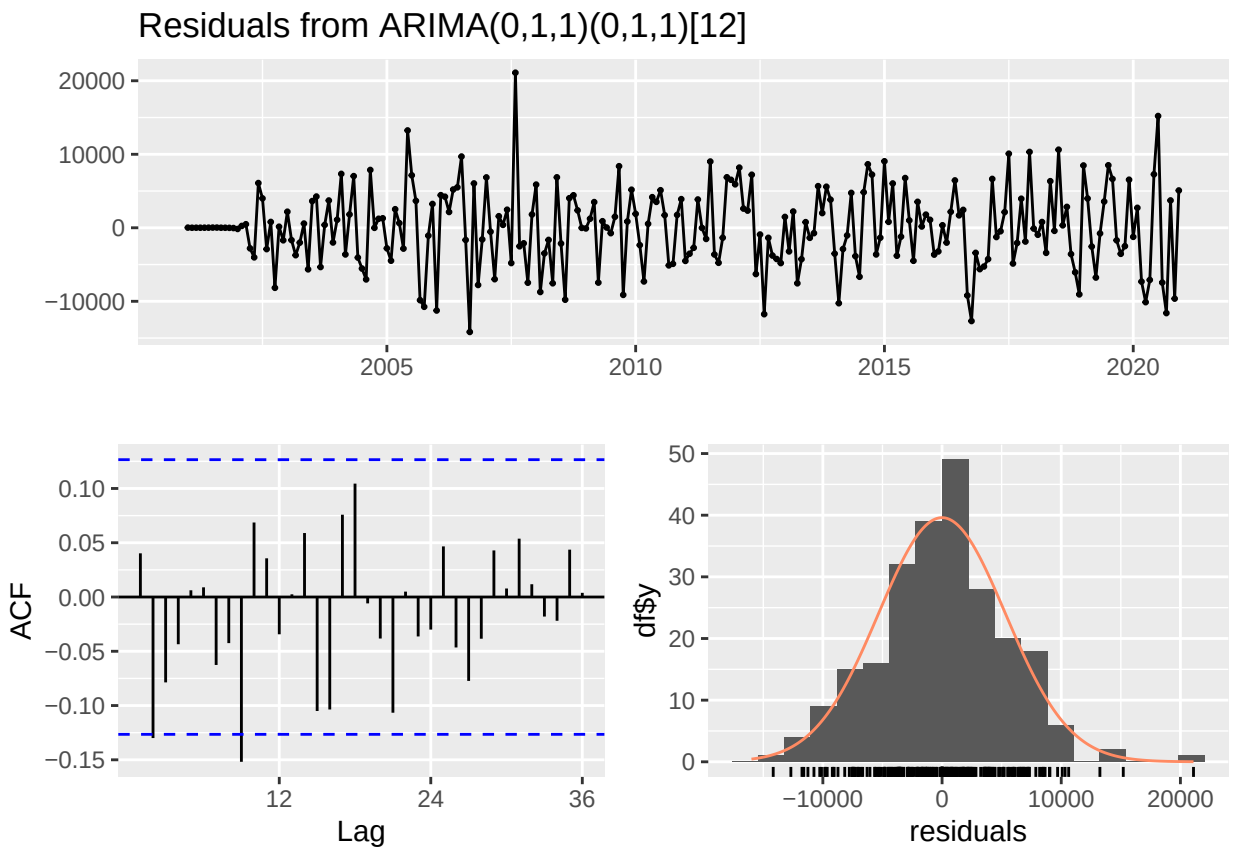
```
## SARIMA _110_011: 4568.855 \n
```

> Answer: SARIMA(0,1,1)(0,1,1) is the best model because it has the lowest AIC of 4565.282 which is consistent with what the ACF/PACF plots suggested that both a non-seasonal MA(1) and seasonal MA(1) term were needed!

Q7 - part Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

```
checkresiduals(Model_011_011)
```



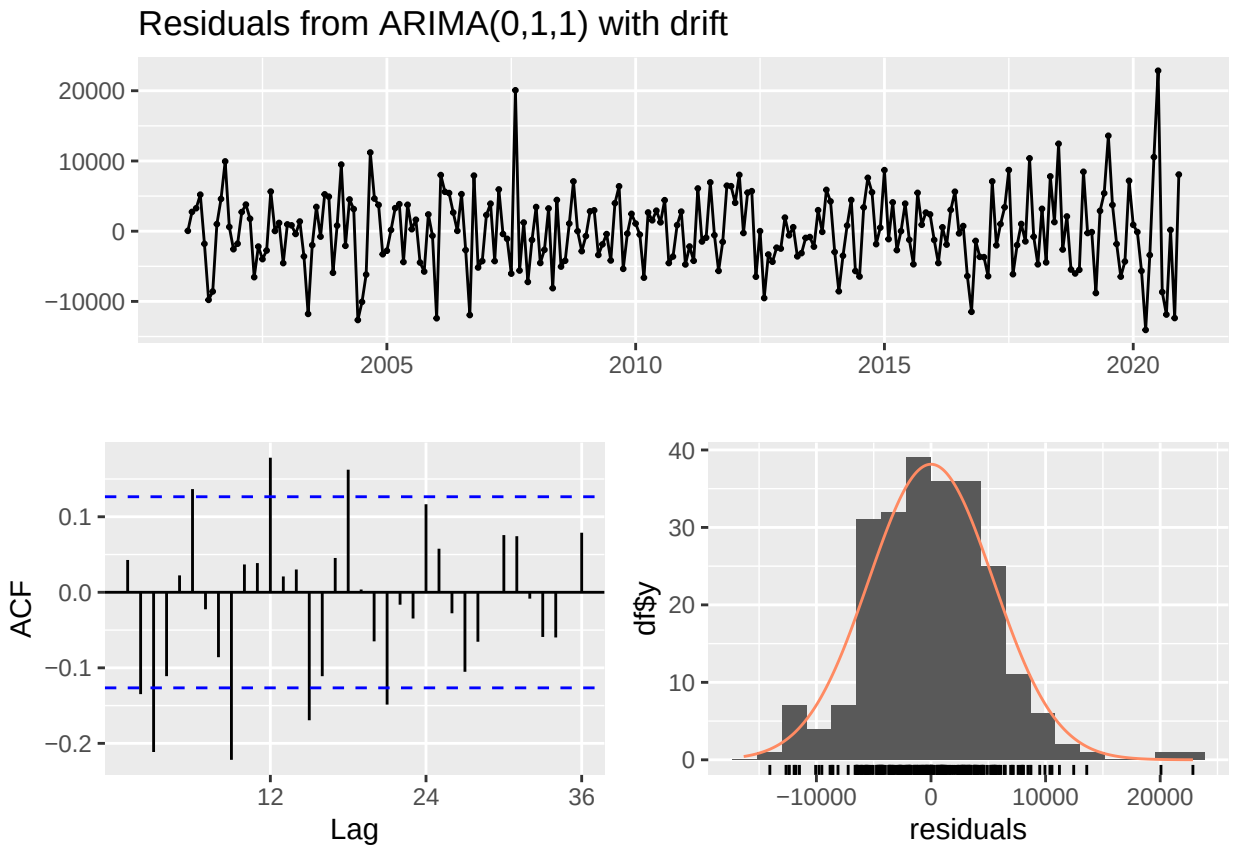

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
## Q* = 30.454, df = 22, p-value = 0.1078
##
## Model df: 2.   Total lags used: 24
```

Answer: very similar to the first part, yes the residuals of (0,1,1)(0,1,1) look like white noise. They fluctate around zero with no visible trend and the variance seems constant over time with a few notable spikes (one likely due to covid). The ACF entirely falls in bounds which is what white noise looks like while the histogram is normally distributed and centered around 0 (though it a little more skewed than the previous). Thus, all centered around zero, normally distributed, and uncorrelated is white noise and shows the model is a decent fit.

Q8

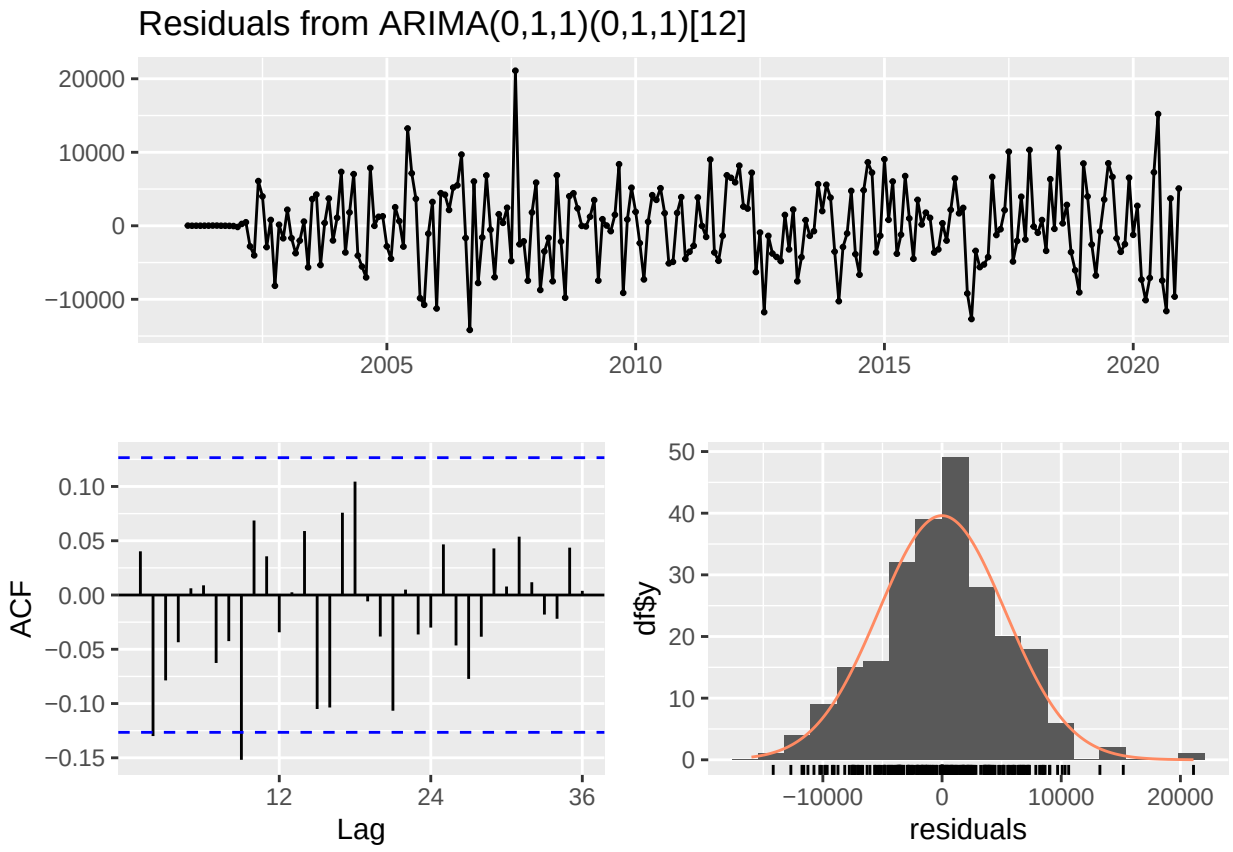
Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

```
checkresiduals(Model_011)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1) with drift
## Q* = 76.2, df = 23, p-value = 1.295e-07
##
## Model df: 1.   Total lags used: 24
```

```
checkresiduals(Model_011_011)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
## Q* = 30.454, df = 22, p-value = 0.1078
##
## Model df: 2.   Total lags used: 24
```

Answer: Both residuals show fluctations around zero with no obvious trend however, the SARIMA seems more stable. The ACF is the most clear indicator of this, with ARIMA having a few spikes out of the bounds vs. the sarima spikes are basically entirely inside the bounds. The cleaner sarima model indicates a better model fit. The sarima histogram also seems a slightly better fit, a bit mor esymmetric and tigther. It not really a fair comparison because they have different input series.

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

```
Model_autofit_deseasonal <- auto.arima(deseasonal_NG)
print(Model_autofit_deseasonal)
```

```
## Series: deseasonal_NG
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

Answer: it specified ARIMA(1,1,1) was the best fit, which is not what i specified in Q4 which I had as ARIMA(0,1,1), this may have been me zooming out too much on my ACF graph.

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?

```
Model_autofit_seasonal <- auto.arima(ts_naturalgas)
print(Model_autofit_seasonal)
```

```
## Series: ts_naturalgas
## ARIMA(1,0,0)(0,1,1)[12] with drift
##
## Coefficients:
##          ar1     sma1     drift
##       0.7416  -0.7026  358.7988
## s.e.  0.0442   0.0557   37.5875
##
## sigma^2 = 27569124:  log likelihood = -2279.54
## AIC=4567.08   AICc=4567.26   BIC=4580.8
```

Answer: The ARIMA(1,0,0)(0,1,1) is the output and does not match my model in Q7 ARIMA(0,1,1)(0,1,1). Once again, i got the p term incorrect which shows i need to relook at how i analyze this.